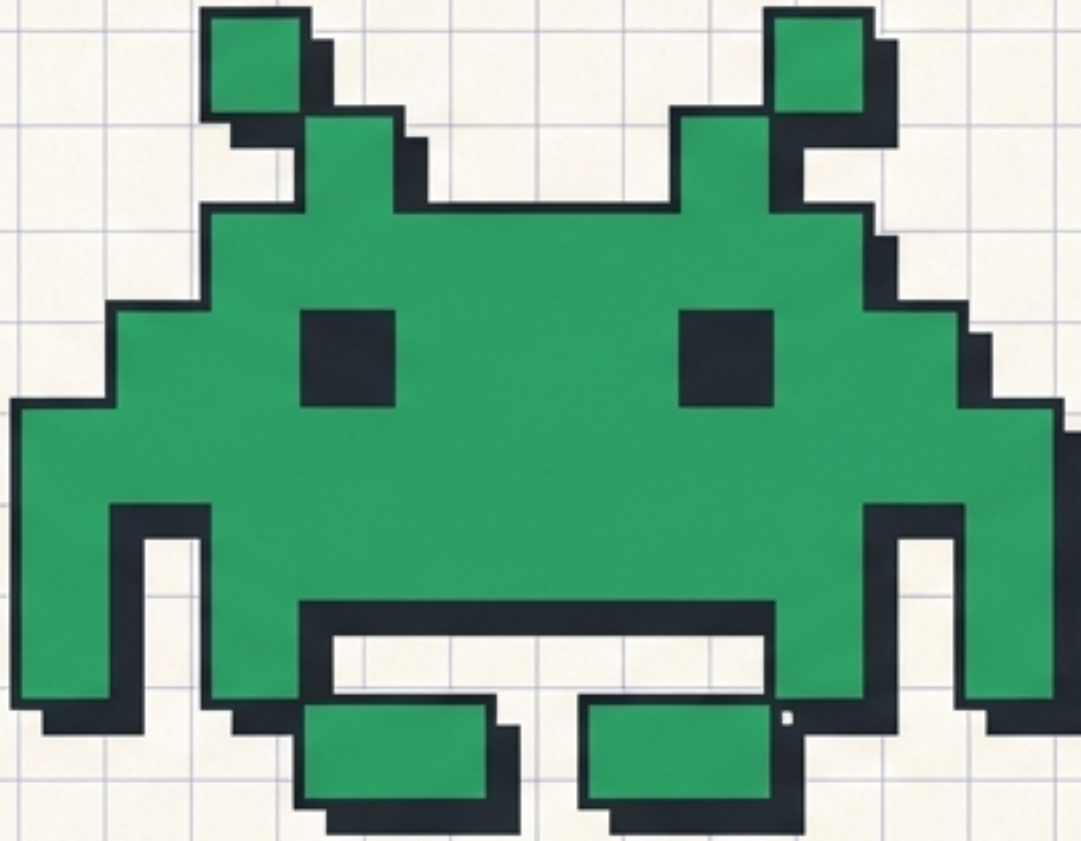


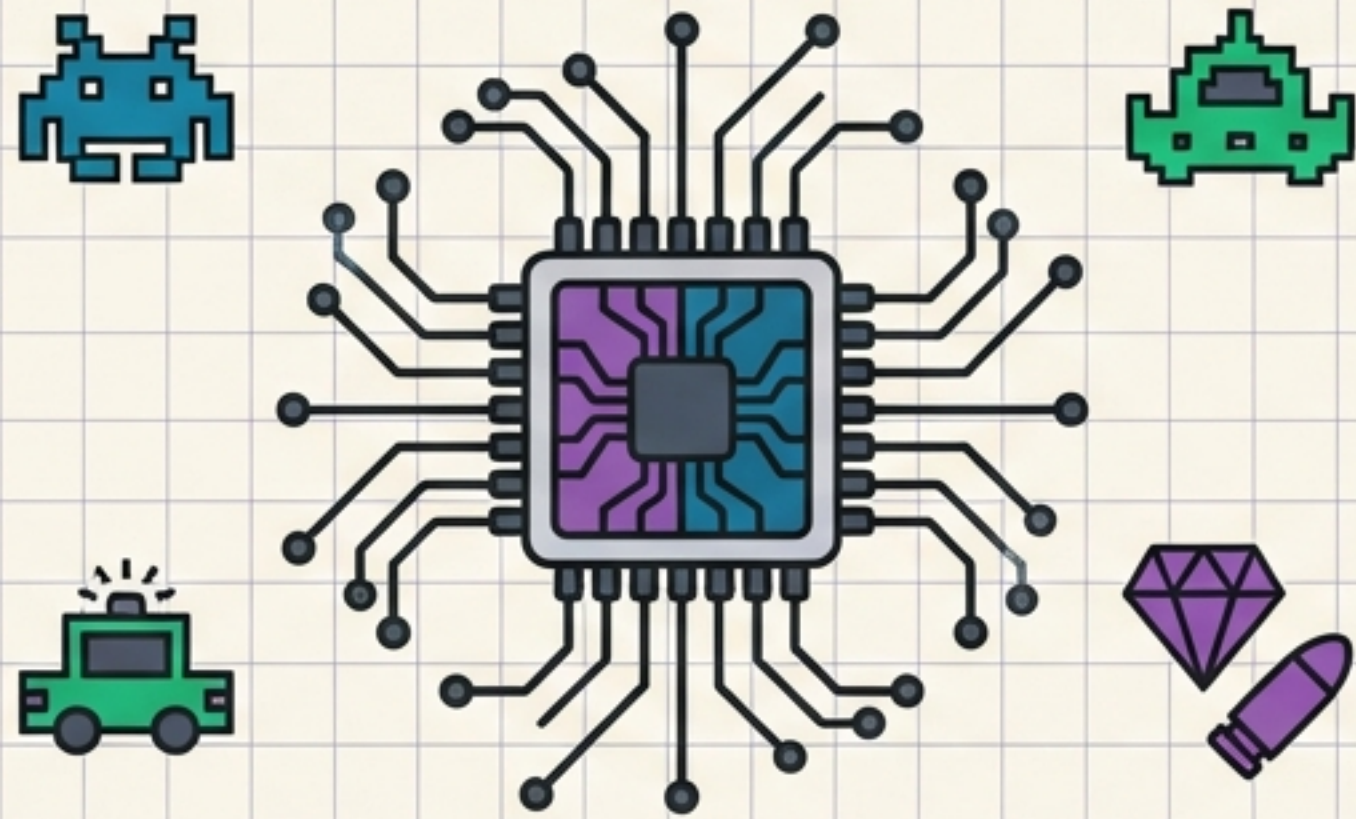
Construyendo Invasores del Espacio



Tu Primera App en **Kotlin** y **Compose**:
De la **lógica pura** a la pantalla en **15 pasos**.

La Arquitectura de Dos Mundos

Juego.kt



El motor calcula silenciosamente dónde está cada pieza. No sabe qué es una pantalla.

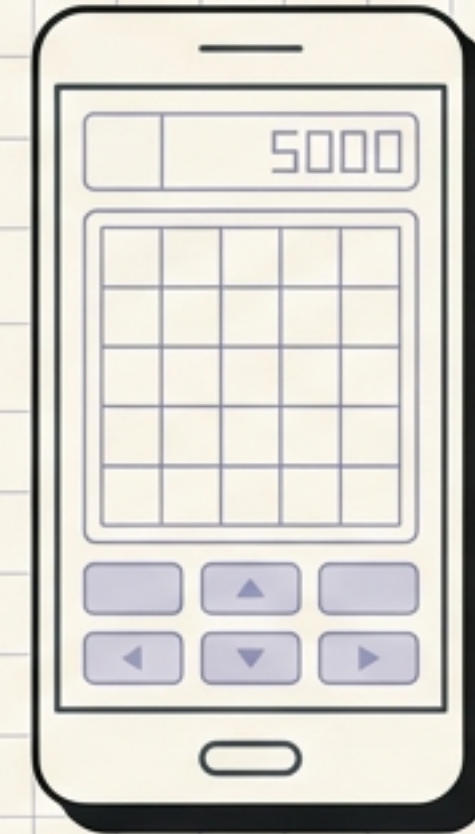
Matemáticas

Reglas

Estado

Kotlin Puro

MainActivity.kt



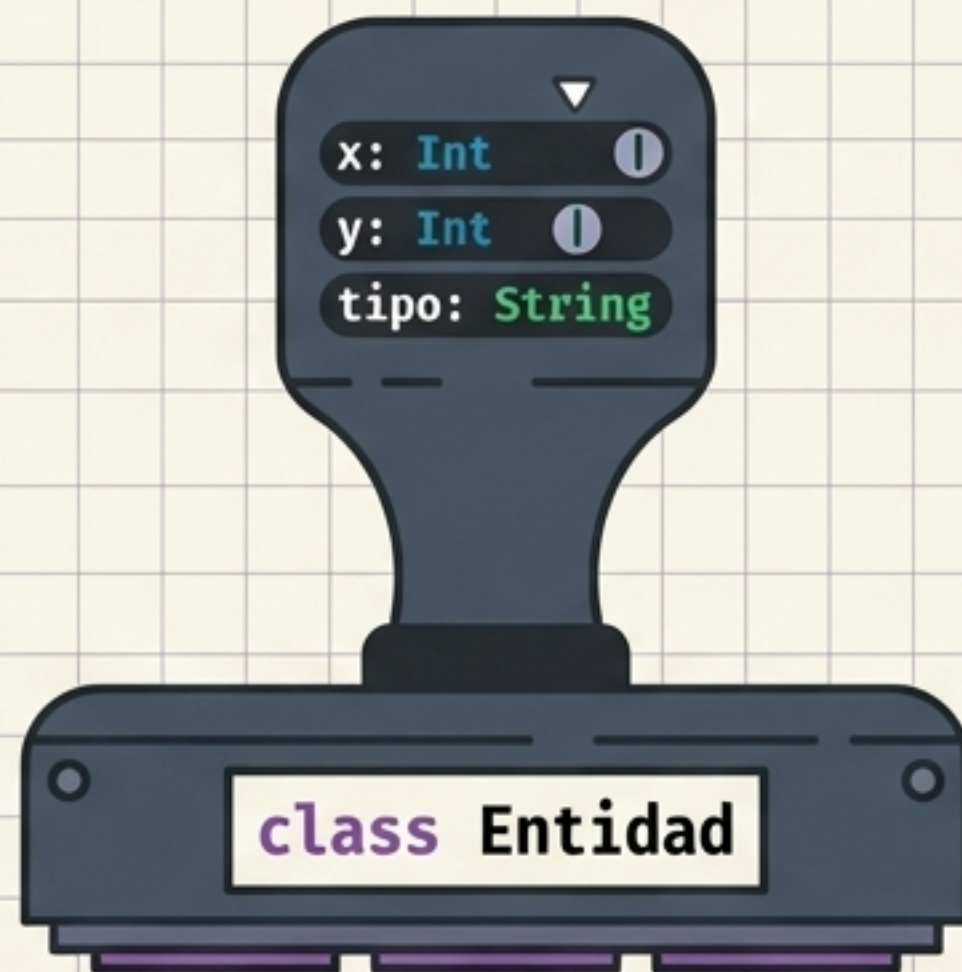
La interfaz gráfica solo dibuja lo que el motor le dicta y escucha los toques del usuario.

Pantalla

Botones

Jetpack Compose

Entidad: El Molde del Universo Físico



Una Clase en Kotlin no es el objeto en sí, es la fábrica que lo crea. Con un solo molde, damos vida a todas las piezas del tablero definiendo su posición (x, y) y su identidad.

`Entidad(5, 15, "NAVE")`



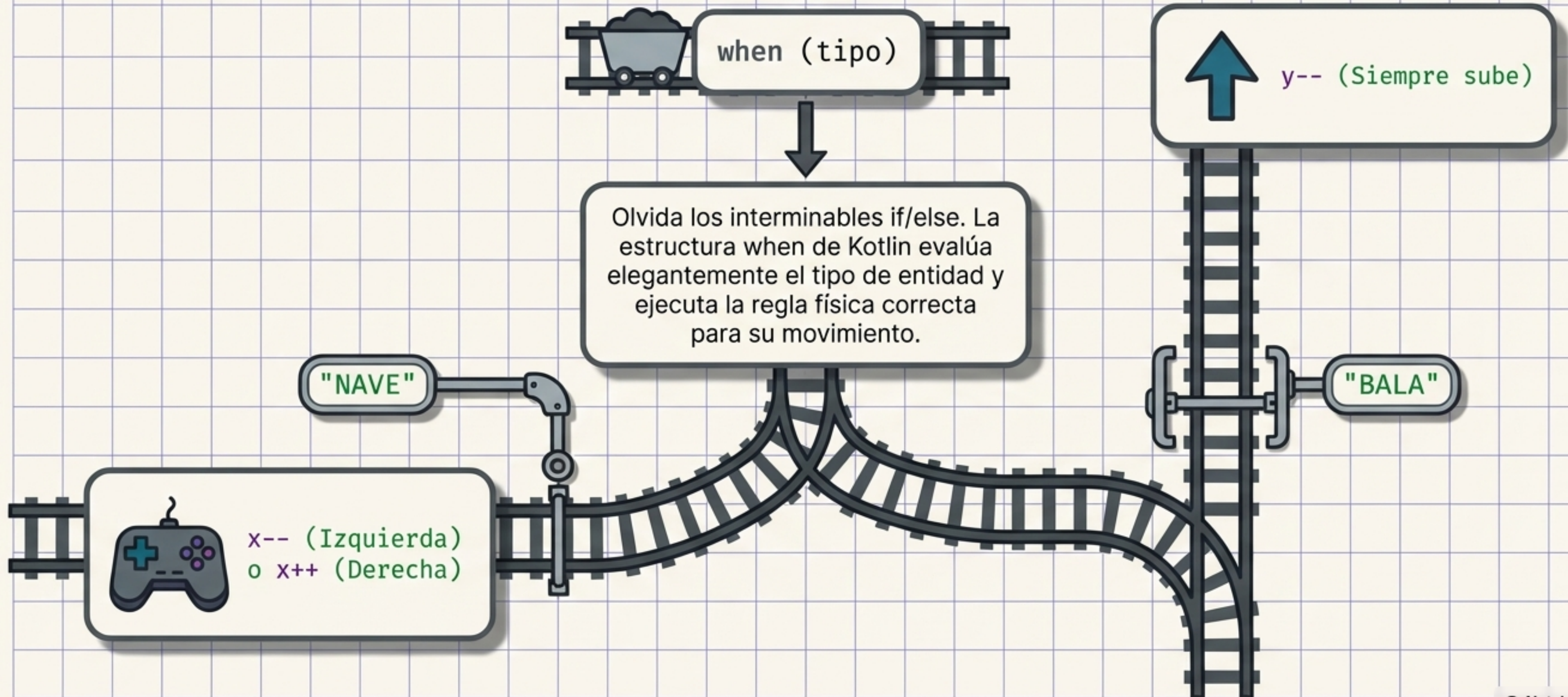
`Entidad(2, 0, "ALIEN")`



`Entidad(5, 14, "BALA")`



Controlando el Caos Numérico con when



El Inventario del Juego: List vs Set

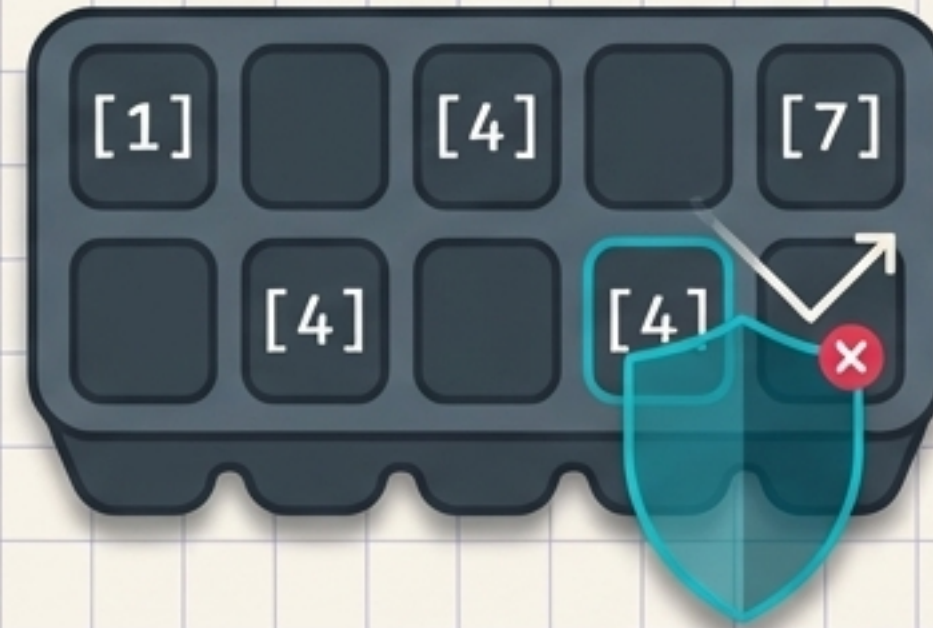
mutableListOf



Permite duplicados.

El inventario general (entidades).
Entran y salen balas dinámicamente.

mutableSetOf

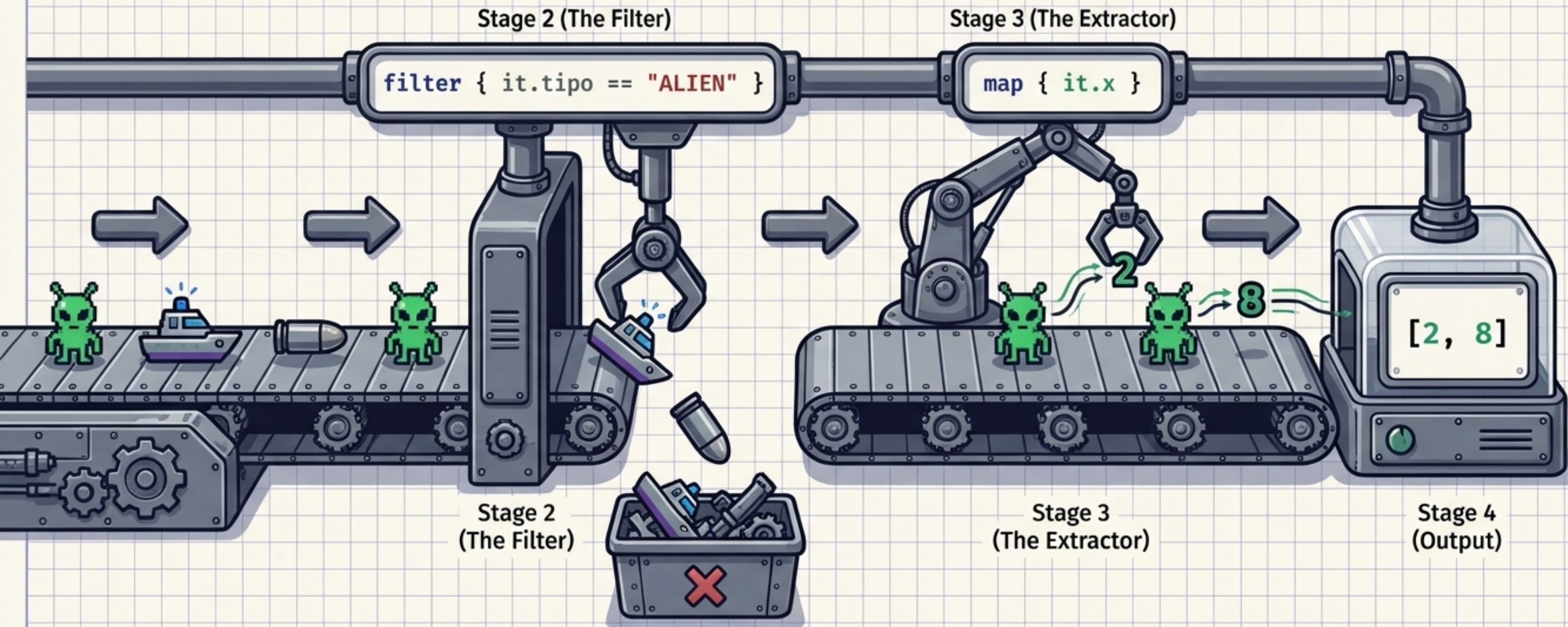


Rechaza duplicados.

Generar oleadas (posicionesUnicas).
Evita que dos aliens nazcan exactamente
en la misma coordenada X.

Magia Funcional: Ordenando a las Tropas

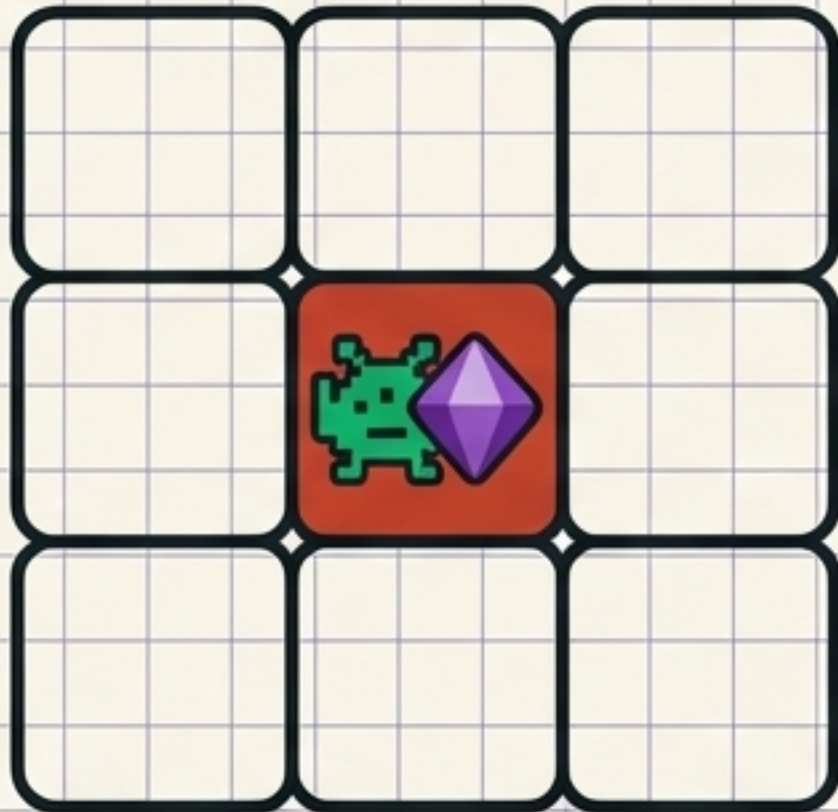
Las funciones de orden superior transforman colecciones enteras en una sola línea de código, extrayendo datos vitales para evitar que los powerups caigan sobre los aliens.



Anatomía Matemática de un Choque

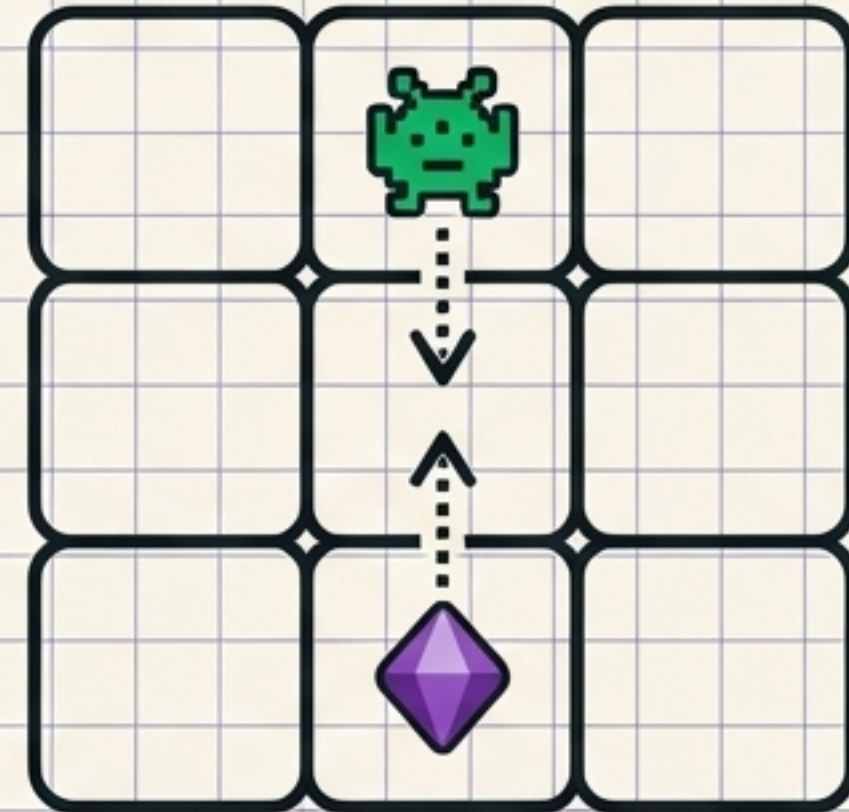
Las colisiones no son magia física; son simple lógica booleana evaluando si dos coordenadas coinciden o se cruzan entre fotogramas.

Escenario A: Choque Directo



```
mismaColumna (&&) choqueDirecto
```

Escenario B: Cruce en el Aire

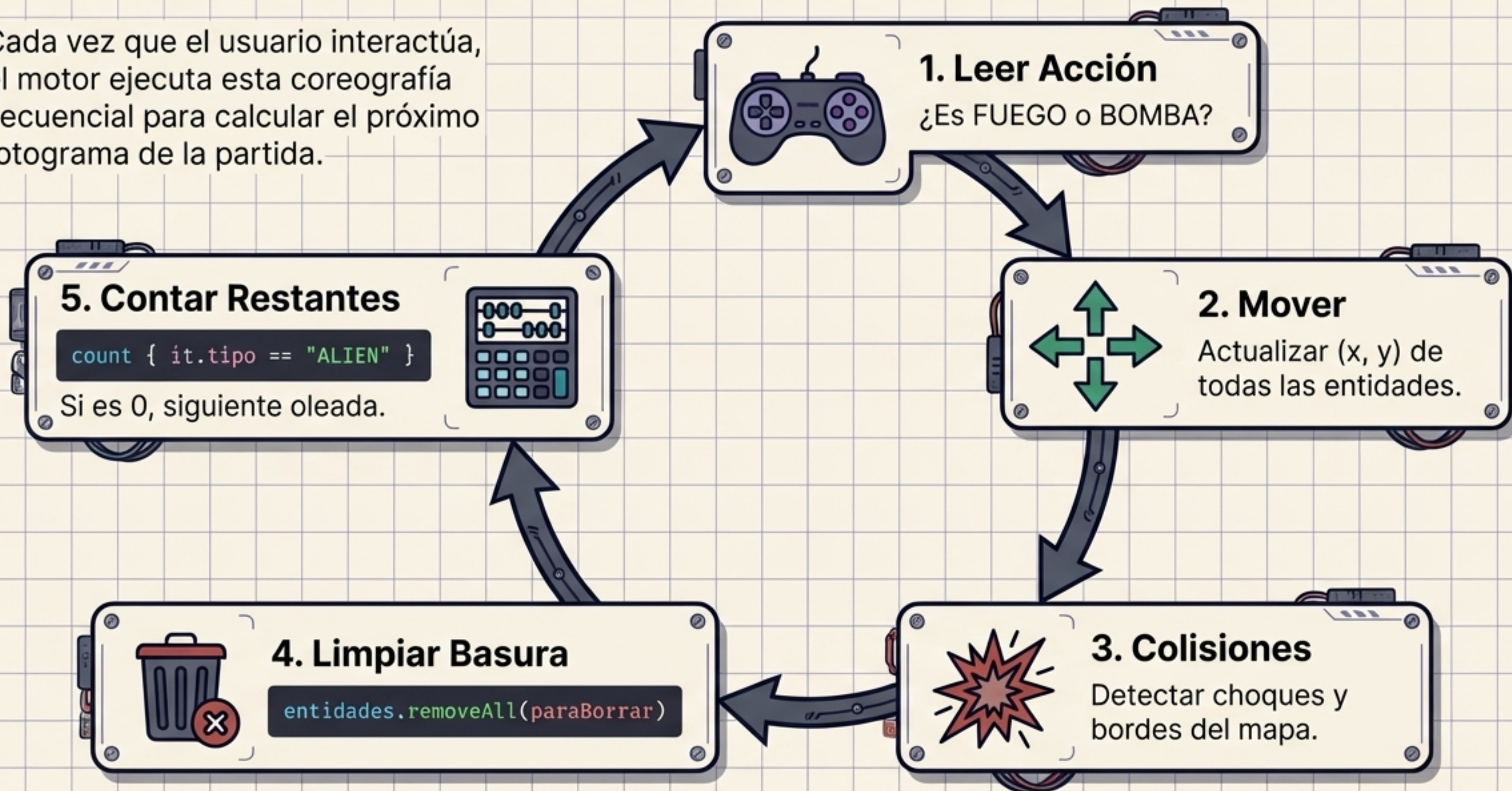


```
mismaColumna (&&) cruceEnElAire
```

```
entidad1.y == entidad2.y + 1
```


El Corazón del Motor: El Ciclo de Turnos

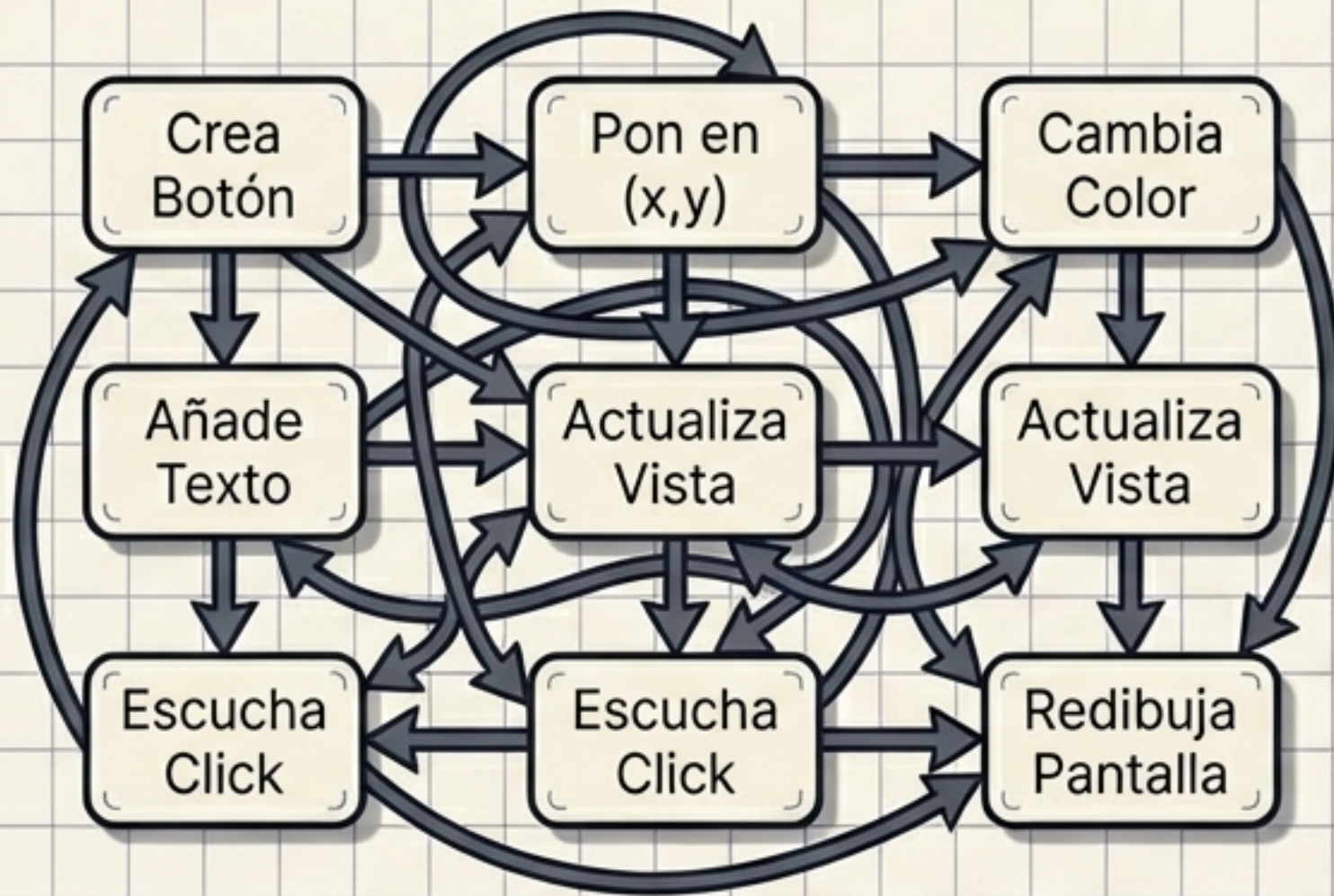
Cada vez que el usuario interactúa, el motor ejecuta esta coreografía secuencial para calcular el próximo fotograma de la partida.



Cruzando la Frontera: Jetpack Compose

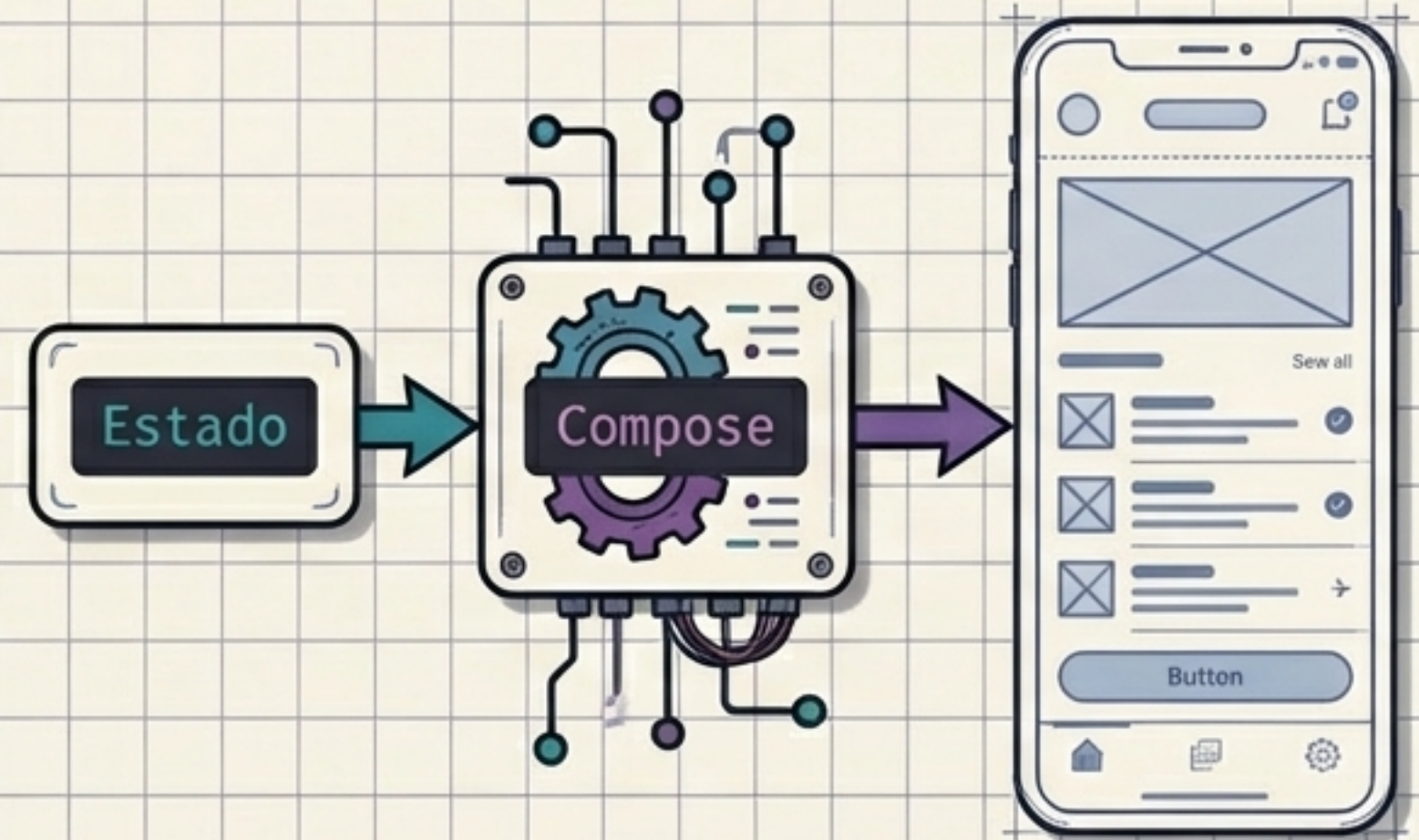
Con `@Composable`, dejamos de dictarle al sistema cómo dibujar paso a paso. Simplemente le describimos qué queremos ver en pantalla, y Compose se encarga de la construcción.

El Paradigma Imperativo (El Pasado)



Dictar la receta paso a paso.

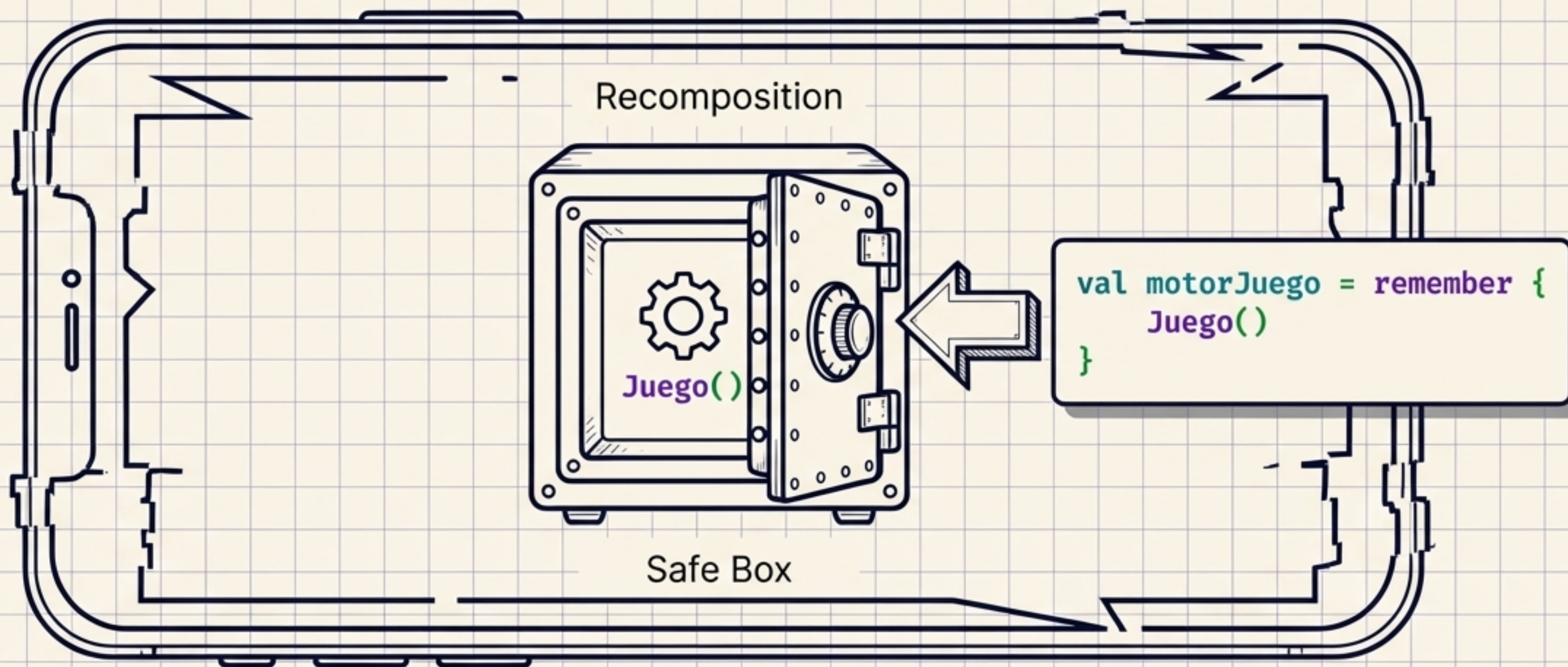
El Paradigma Declarativo (@Composable)



Entregar el pedido en el restaurante.

La Memoria Muscular: Protegiendo el Estado

Compose redibuja la pantalla docenas de veces por minuto. Es amnésico por diseño. Usamos remember como una caja fuerte para que nuestro progreso sobreviva a cada refresco de pantalla.



Forzando el Redibujado: El Estado Artificial

Al cambiar un valor de estado (State), Compose detecta la alteración y dispara automáticamente la actualización de toda la interfaz gráfica con los datos más recientes.

```
var refrescar by remember  
    { mutableStateOf(0) }
```

refrescar++

Señal de Recomposición

PUNTOS: 100

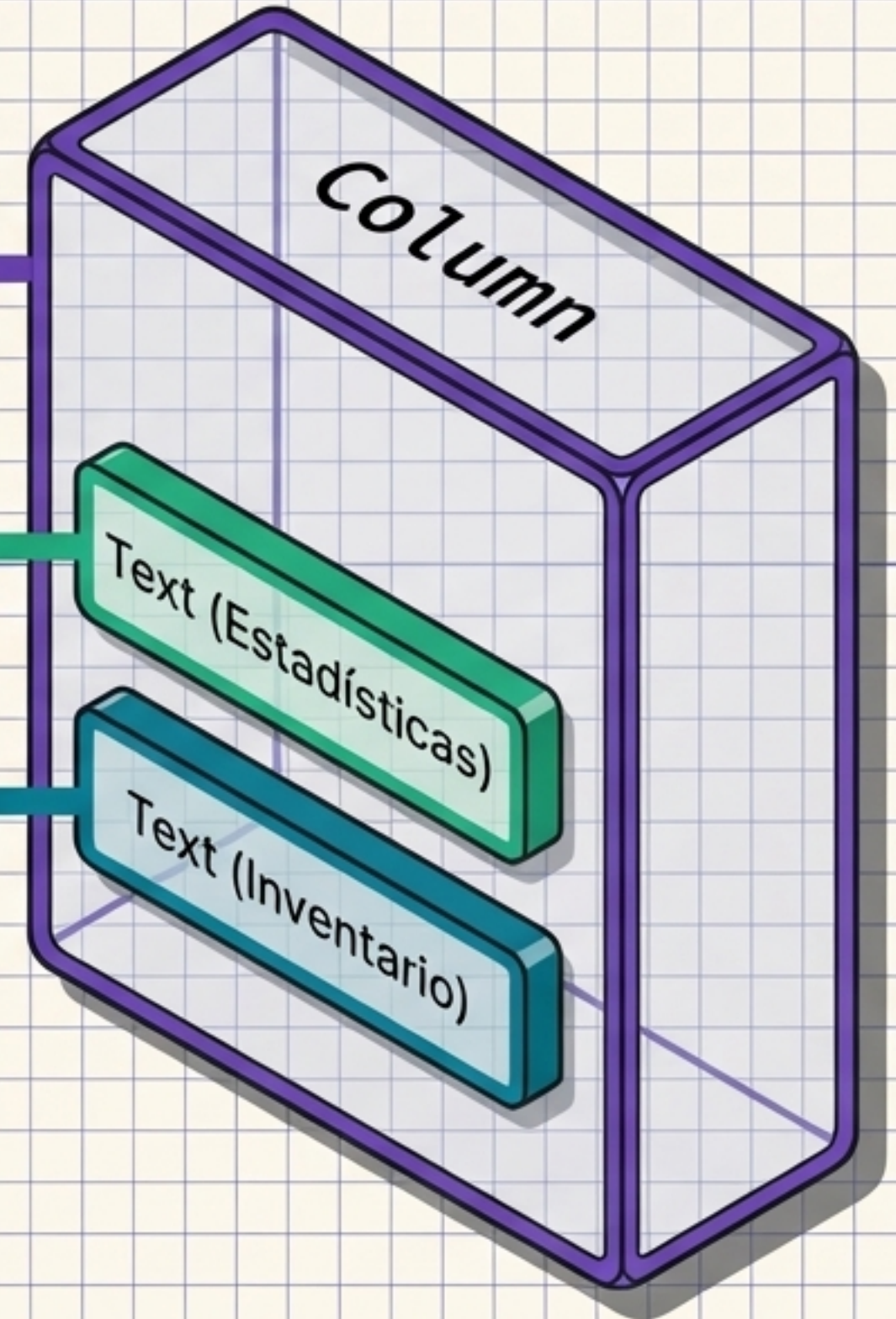
VIDAS: 3 ❤️

ALIENS: 2 🚗

Arquitectura de Diseño: Apilando Bloques

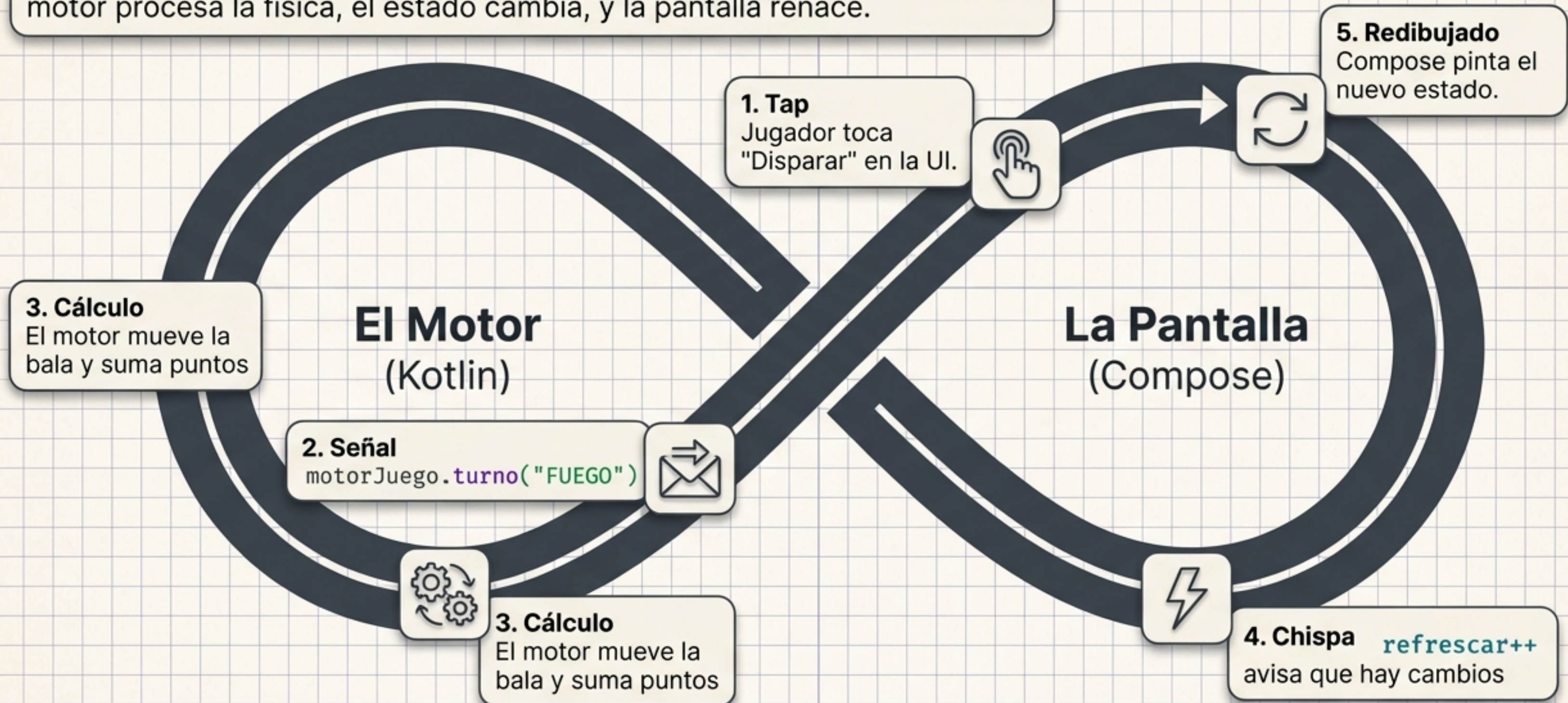
El diseño moderno de interfaces es puro Tetris. Agrupamos elementos visuales **apilándolos verticalmente** con **Column** o alineándolos **horizontalmente** con **Row**.

```
Column(  
  horizontalAlignment = CenterHorizontally  
) {  
  Text(text = "PUNTOS 500 | VIDAS 3")  
  Text(text = "BOMBAS DISPONIBLES: 1")  
}
```



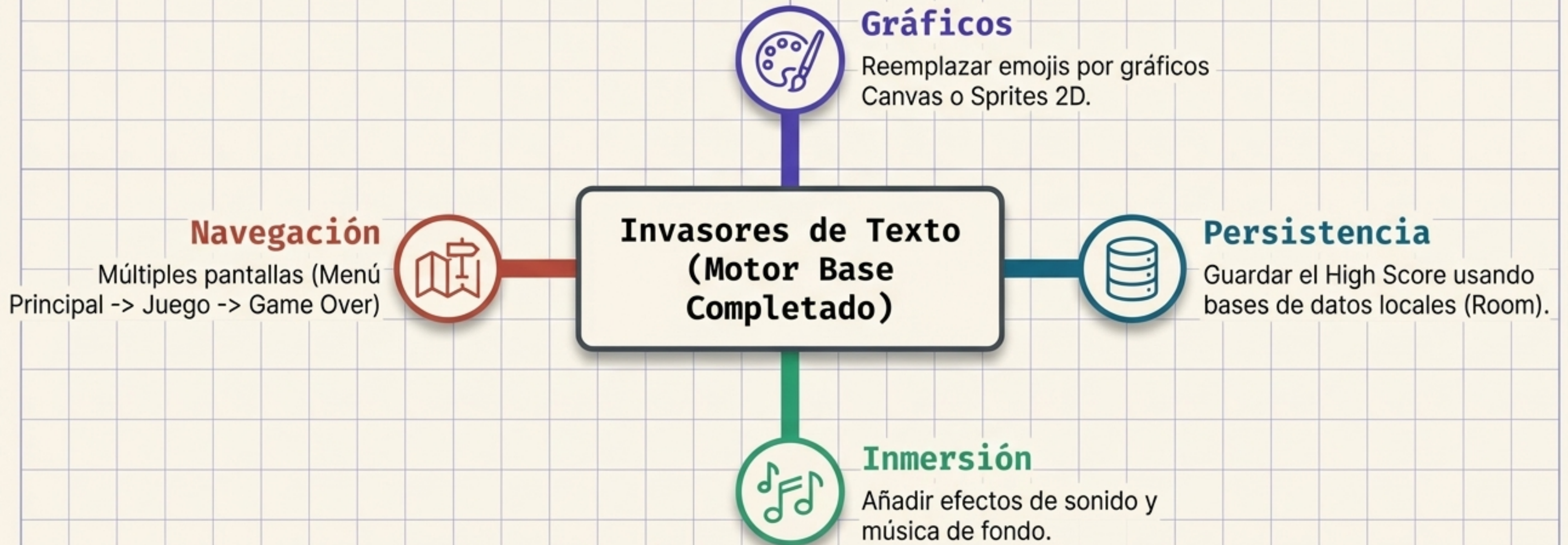
El Hilo Conductor: El Bucle Infinito

La integración perfecta del Modelo-Vista. La interfaz capta la intención, el motor procesa la física, el estado cambia, y la pantalla renace.



Mapa de Habilidades: Siguiendo Pasos

Tienes el motor base funcionando. Ahora es el momento de expandir tu arsenal como desarrollador Android. ¿Qué rama desbloquearás primero?





GAME OVER (Press Start)

> Has superado el tutorial. En solo un proyecto, has dominado:



Arquitectura limpia y separación lógica.



Colecciones mutables y funciones de orden superior en Kotlin.



Estados reactivos e interfaces declarativas con Jetpack Compose.

■ El editor de código es tuyo. Inicia tu próximo proyecto.